

Implement A star Algorithm for any game search problem

A* Algorithm Implementation in Python

Graph represented as adjacency list

```
g = {  
    'A': [('B', 6), ('F', 13)],  
    'B': [('C', 3), ('D', 2)],  
    'C': [('D', 1)],  
    'D': [('E', 8)],  
    'E': [('J', 5), ('I', 5)],  
    'F': [('G', 1), ('H', 7)],  
    'G': [('I', 3)],  
    'H': [('I', 2)],  
    'I': [('J', 3), ('E', 5)],  
    'J': []  
}
```

Heuristic values

```
h = {  
    'A': 10,  
    'B': 8,  
    'C': 5,  
    'D': 7,  
    'E': 3,  
    'F': 6,  
    'G': 5,  
    'H': 3,  
    'I': 1,
```

```
'J':0  
}
```

```
# Start and Goal node
```

```
start = 'A'
```

```
goal = 'J'
```

```
# OPEN list
```

```
open = [start]
```

```
# Cost from start node
```

```
g_cost = {start:0}
```

```
# Parent dictionary
```

```
parent = {start:start}
```

```
while open:
```

```
    # Find node with minimum  $f(n)=g(n)+h(n)$ 
```

```
    n = open[0]
```

```
    for v in open:
```

```
        if g_cost[v] + h[v] < g_cost[n] + h[n]:
```

```
            n = v
```

```
    # Goal node found
```

```
    if n == goal:
```

```
        path = []
```

```
        while parent[n] != n:
```

```
    path.append(n)
    n = parent[n]

    path.append(start)
    path.reverse()

    print("Optimal Path:")
    print(" -> ".join(path))

    print("Total Cost =", g_cost[goal])
    break

# Remove current node from OPEN list
open.remove(n)

# Expand neighbouring nodes
for (m, cost) in g[n]:

    new_cost = g_cost[n] + cost

    # Update cost if better path found
    if m not in g_cost or new_cost < g_cost[m]:

        g_cost[m] = new_cost
        parent[m] = n

    open.append(m)
```

A* (A-Star) Algorithm is a heuristic search algorithm used to find the shortest and most optimal path between a start node and a goal node. It is widely used in games, maps, robotics, and AI pathfinding problems.

The algorithm works using:

- $g(n)$ = actual cost from start node to current node
- $h(n)$ = heuristic value (estimated cost to reach goal)
- $f(n) = g(n) + h(n)$

$$f(n)=g(n)+h(n)$$

The node having the smallest $f(n)$ value is selected for expansion. This helps the algorithm reach the goal faster than normal search algorithms.

Explanation of the Program:

1. Graph Creation

The graph is represented using a dictionary where each node stores neighboring nodes and edge costs.

Example:

```
'A':[('B',6),('F',13)]
```

This means:

- $A \rightarrow B$ with cost 6
- $A \rightarrow F$ with cost 13

2. Heuristic Function

The heuristic dictionary stores estimated distance from each node to goal node J.

Example:

```
h = {'A':10, 'B':8, ..., 'J':0}
```

3. OPEN List

OPEN list contains nodes that are available for exploration.

```
open = [start]
```

4. g_cost Dictionary

Stores shortest distance from start node.

Example:

```
g_cost = {start:0}
```

5. Parent Dictionary

Used to store previous node so the final path can be reconstructed.

6. Selecting Best Node

The algorithm checks all nodes in OPEN list and selects the node having minimum:

$g_cost[v] + h[v]$

7. Goal Test

If current node becomes goal node J, the algorithm stops and prints:

- Optimal Path
- Total Cost

8. Path Reconstruction

Using parent dictionary, the algorithm traces path from goal back to start.

Output:

Optimal Path:

A -> F -> G -> I -> J

Total Cost = 20

Advantages of A* Algorithm:

- Finds shortest path efficiently
- Faster than BFS and DFS
- Uses heuristic to reduce searching time
- Commonly used in GPS and game AI

Applications:

- Google Maps navigation
- Video game pathfinding
- Robot navigation
- Network routing systems